ExpenseEase — Project Report

Course: Software Engineering & Project Management (CSN15101)
Section: E
Instructor: Dr. Satarupa Chakrabarti
Institute: Motilal Nehru National Institute of Technology Allahabad
Submission Date: 16th October 2026

Team Members

Name	Registration Number
Shreeya Srivastava	20243267
Shraddha Sharma	20243266
Rudransh Pratap Singh	20243243
Sachit Jain	20243245

---

1. Problem Statement and Motivation

1.1 Background

Managing shared expenses within a group — whether it is a college trip, shared accommodation, or a project team dinner — is inherently messy. The typical approach is a WhatsApp thread of receipts and informal “you owe me” messages, which quickly becomes untrackable. People forget who paid what, calculations are done mentally and are prone to error, and settling debts often requires multiple rounds of back-and-forth negotiation.

The problem grows with the size and duration of the group. A weekend trip with six friends may involve fifteen or more separate payments made by different people, for different subsets of the group, in different amounts. By the end of the trip no single person can reliably reconstruct who has contributed what. Disagreements about money, even small ones, strain friendships and make people reluctant to share costs the next time.

1.2 The Core Problem

The underlying need is simple: a group wants to know *who owes whom, and how much*, and wants to clear those debts with as few payments as possible. Three specific failure modes make the informal approach unreliable:

- **Missing records.** Receipts and payment messages are scattered across chats, and there is no authoritative list of what was spent.
- **Arithmetic errors.** Dividing an amount unevenly among a subset of the group, handling remainders, and netting multiple expenses by hand is error-prone.
- **Unverifiable outcomes.** Even when a tool produces a number, members cannot easily check how it was derived, which erodes trust in the result.

1.3 Limitations of Existing Solutions

Applications such as Splitwise exist, but they are either locked behind paywalls for core features, require app installation, or are overkill for a student group that simply wants to know who owes whom and how much. There is also a trust problem: when an app calculates balances, users often cannot verify how those numbers were arrived at.

Approach	Strengths	Weaknesses for the target audience
WhatsApp thread + mental maths	Zero setup; everyone already has it	No structure, no audit trail, frequent errors, hard to settle
Spreadsheet	Flexible; transparent formulas	Manual data entry, no access control, awkward on phones, easy to corrupt
Commercial expense apps	Polished, feature-rich	Paywalled core features, mandatory installation, opaque calculations
ExpenseEase	Browser-based, free, published formula, deterministic, minimised transfers	Smaller feature set (by design); INR only

Table 1: Comparison of approaches to shared-expense management

1.4 Proposed Solution

ExpenseEase is a web-based expense-splitting application built specifically for small, trust-based groups such as student flatmates, trip groups, or project teams. It records shared expenses, computes individual balances transparently using a published formula, and minimises the number of cash transfers needed to clear all debts. Every calculation is deterministic — the same input always produces the same output — so any member can manually verify the results.

1.5 Motivation

The motivation for building this system came from a real pain point experienced by the team itself during inter-college events and shared accommodation. The project is also an opportunity to put software engineering principles into practice in a meaningful, self-contained domain: eliciting requirements, choosing a lifecycle model, designing a layered architecture, enforcing quality through automated tests, managing risk, and tracking progress across a four-person team.

---

2. Project Objectives

The primary goal of ExpenseEase is to provide a group with a single, honest ledger for shared expenses. Specifically, the system is designed to achieve the objectives below. Each objective is paired with the way in which its success is measured, so that progress can be judged objectively rather than by impression.

#	Objective	How it is achieved	How success is verified
1	Eliminate manual calculation errors	All split arithmetic is performed server-side using integer paise	Automated unit tests assert shares sum exactly to the total
2	Support multiple split strategies	Equal, exact-amount and percentage-based splits	Per-strategy test suites including edge cases
3	Produce verifiable balances	Documented formula; invariant that all balances in a group sum to zero	Invariant used as a test oracle on every balance test
4	Minimise settlement transactions	Greedy debt-simplification algorithm	Output has at most n-1 transfers for n non-zero members

#	Objective	How it is achieved	How success is verified
5	Maintain a full audit trail	Every expense and settlement is logged with who, what and when	Activity feed reproduces history of each group
6	Be accessible without installation	React SPA served to any modern browser	No native app required; works on desktop and mobile browsers
7	Protect member data	bcrypt hashing, JWT on every endpoint, membership-scoped data	Unauthorised-access tests return 401/403

Table 2: Project objectives and verification criteria

---

3. Scope of the Project

3.1 Features Included

Feature	Description
User registration and login	Email + password authentication with JWT session tokens (7-day expiry)
Group creation	Any authenticated user can create a group and becomes its administrator
Member management	Group admin can add registered users by email address
Expense recording	Members record expenses with description, amount, category, date, payer, and split type
Equal split	Amount divided equally; remainder paise distributed deterministically by user ID
Exact split	Each participant’s share entered manually; validated to sum to total
Percentage split	Each participant’s percentage entered; validated to total 100; shares computed server-side
Balance calculation	Net balance per member computed as total paid minus total owed, adjusted for settlements
Debt simplification	Greedy algorithm reduces the number of cash transfers needed to clear all debts
Settlement recording	Any member can record a payment between two group members
Activity log	Every expense and settlement action is timestamped and logged per group
Cross-group dashboard	Logged-in user sees total “you owe” and “owed to you” across all groups
Expense editing	Expense creator or group admin can edit any field, including split details
Expense deletion	Expense creator or group admin can delete an expense

Table 3: Features included in the delivered scope

3.2 Features Excluded

The following were considered but deliberately excluded from the mid-semester deliverable. Each exclusion was made to keep the system small enough to be verified thoroughly; the rationale is recorded so that future maintainers understand that these are conscious omissions rather than oversights.

Excluded feature	Reason for exclusion
Password reset	No email delivery mechanism is in scope; a placeholder message is shown
Notifications	No email or push notification system
File / receipt attachments	Expenses are text-only; storage and upload security are out of scope

Excluded feature	Reason for exclusion
Currency support beyond INR	All amounts are stored in paise (Indian Rupees only)
Recurring expenses	No auto-repeat functionality
Expense categories as a dropdown	Category is a free-text field; no enforcement of a fixed taxonomy
Group deletion or archiving	Groups persist indefinitely
Mobile application	The system is web-only; no native Android or iOS app
Offline mode	No service worker or local-first data strategy

Table 4: Features deliberately excluded

3.3 Assumptions and Constraints

Assumptions

- All group members must be registered users before they can be added to a group; adding by name alone is not supported.
- All monetary amounts are in Indian Rupees. Amounts are entered in rupees (e.g., Rs. 150.00) and stored as integers in paise (15000) to avoid floating-point rounding.
- Percentage splits accept whole-number percentages only (e.g., 33% is valid; 33.3% is not).
- The group administrator who creates the group is the only user who can add new members.
- Settlements do not automatically close or archive; balances accumulate across all expenses and settlements.

Constraints

- The system requires Node.js 20+ and a PostgreSQL 16 database.
- The Docker Compose deployment is the supported production setup; local development requires manual environment variable configuration.
- There is a rate limit of 20 login or registration attempts per minute per IP to prevent brute-force attacks.
- JWT tokens expire after 7 days; there is no token refresh mechanism in the current version.
- The project must be completed within the semester timeline by a team of four students, which bounds the feature set.

---

4. Stakeholders

Stakeholders were identified early so that requirements could be traced to a real need. The influence rating reflects how strongly each party can shape the outcome of the project.

Stakeholder	Interest	Influence	How the project addresses them
Group members	Accurate shared costs	High	Deterministic splits, transparent formula, audit trail
Group admins	Membership and data oversight	High	Admin-only member management; edit and delete rights on expenses
Course evaluator	Evidence of engineering practice	High	Traceable requirements, tests, project-management artefacts
Development team	Deliverable quality and learning	High	Iterative delivery, code review, quality gates

Stakeholder	Interest	Influence	How the project addresses them
Future maintainers	Clear design and tests	Medium	Layered architecture, 179 automated tests, documented design

Table 5: Stakeholder analysis

Group members and admins are the primary end users. Their requirements dominated the functional specification, particularly the need for correct and explainable balances. The course evaluator and the development team are internal stakeholders whose interests drive the non-functional emphasis on testing, documentation and process discipline.

---

5. Requirements Specification

5.1 Functional Requirements

Functional requirements follow the pattern *"When [condition], the system shall [behaviour], with [measurable result]."* The full SRS is maintained in the repository's docs folder; the table below summarises it.

ID	Requirement
FR-AUTH-01	The system shall allow a registered user to log in with valid credentials and reject invalid credentials with an error message.
FR-AUTH-02	The system shall allow a new user to register with a unique email address and a password, storing only a salted hash.
FR-GRP-01	The system shall let an authenticated user create a group and become its administrator.
FR-GRP-02	The system shall let a group admin add registered users to the group by email address.
FR-EXP-01	The system shall let a group member record an expense with payer, amount, description, category, date and participants.
FR-EXP-02	For an equal split, the system shall divide the amount among participants so that the shares sum exactly to the total.
FR-EXP-03	For an exact split, the system shall reject the expense if the entered shares do not sum to the total.
FR-EXP-04	For a percentage split, the system shall reject the expense if the percentages do not sum to 100.
FR-EXP-05	The system shall let the expense creator or group admin edit or delete an expense.
FR-BAL-01	The system shall compute each member's net balance in a group as total paid minus total owed, adjusted for settlements.
FR-BAL-02	The system shall produce a simplified list of transfers that clears all balances.
FR-SET-01	The system shall record a settlement between two members and update both balances.
FR-ACT-01	The system shall log every expense and settlement action in an activity feed.
FR-DSH-01	The system shall show a logged-in user the total they owe and are owed across all groups.

Table 6: Functional requirements

5.2 Non-Functional Requirements

ID	Category	Requirement
NFR-SEC-01	Security	Passwords shall be stored using bcrypt salted hashing, never as plain text.

ID	Category	Requirement
NFR-SEC-02	Security	Every API endpoint other than registration and login shall require a valid JWT; group data shall be visible to members only.
NFR-SEC-03	Security	Login and registration shall be rate-limited to 20 attempts per minute per IP.
NFR-REL-01	Reliability	Balance calculation shall be deterministic: identical inputs always give identical outputs.
NFR-REL-02	Reliability	The sum of all net balances in a group shall always equal zero (balance invariant).
NFR-VAL-01	Validation	All request bodies shall be validated against declarative Zod schemas before any business logic runs.
NFR-PERF-01	Performance	Balance views shall load within 2 seconds for a group of up to 50 members and 1,000 expenses.
NFR-MNT-01	Maintainability	Business logic shall be separated from UI and data access to support unit testing.
NFR-DEP-01	Deployability	The system shall run from a Docker Compose definition with a Dockerized PostgreSQL database.

Table 7: Non-functional requirements

5.3 Representative User Stories

- **As a trip organiser**, I want to record that I paid for the hotel on behalf of four friends, so that everyone's share is calculated without me doing the maths.
- **As a flatmate**, I want to split the electricity bill by percentage because we use different amounts, so that the split reflects our agreement.
- **As a group member**, I want to see the exact formula behind my balance, so that I can trust the number and verify it myself.
- **As a debtor**, I want a short list of who I should pay, so that I make as few transfers as possible.
- **As a group admin**, I want to correct a wrongly entered expense, so that the ledger stays accurate.

5.4 Traceability

Requirements are traced forward to design elements and tests, so that every requirement can be shown to be both implemented and verified. An excerpt of the Requirements Traceability Matrix (RTM) follows.

Need	Requirement	Design element	Test
Split bills fairly	FR-EXP-02	Equal-split service	Unit: shares sum to total
Prevent wrong entries	FR-EXP-03 / 04	Split validators (Zod + service)	Unit: invalid sums rejected
Know who owes what	FR-BAL-01	Balance service	Unit: net balances sum to zero
Clear debts	FR-SET-01	Settlement service	Integration: balances update
Fewer transfers	FR-BAL-02	Debt-simplification routine	Unit: at most n-1 transfers; all balances reach 0
Keep data private	NFR-SEC-02	JWT middleware + membership check	API: unauthorised requests rejected

Table 8: Requirements traceability matrix (excerpt)

6. System Design

6.1 Architectural Overview

The system is a browser-based SPA (Single Page Application) that communicates with an Express REST API. The backend uses Prisma for database access, and balances are computed dynamically from expenses and settlements to prevent stale data. The architecture is layered so that responsibilities are separated, giving high cohesion within each layer and low coupling between layers.

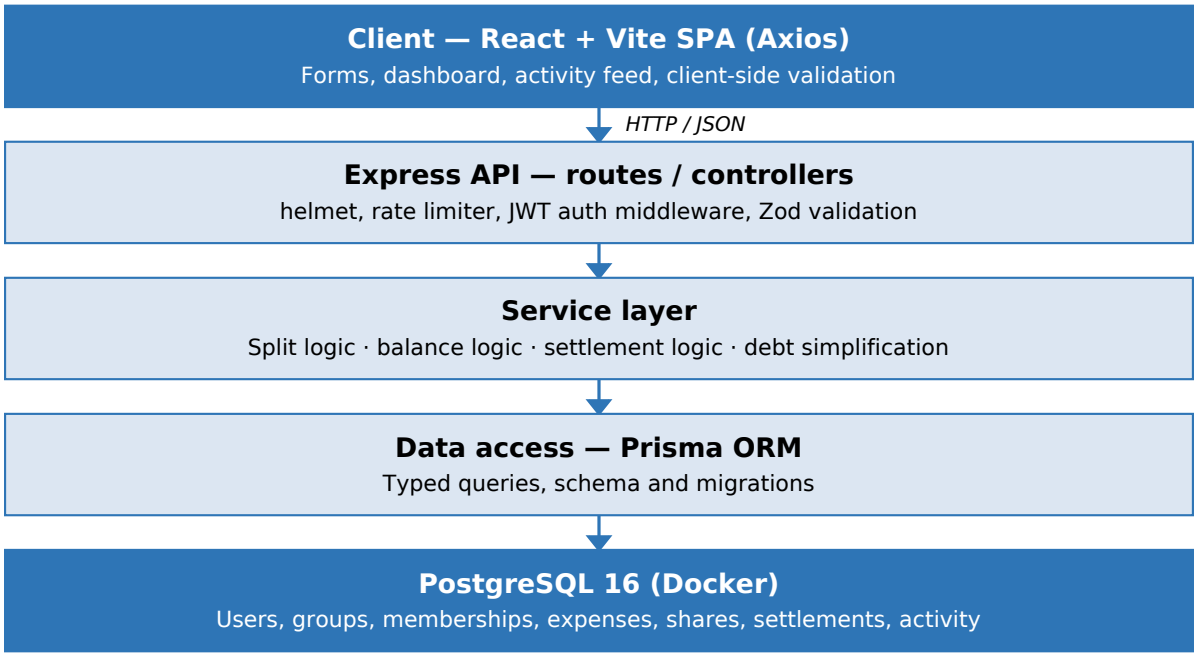


Figure 1: Layered architecture of ExpenseEase

- **The UI never touches the database directly.** All persistence goes through the API.
- **Split and balance logic** lives in the service layer, independent of HTTP and the database, so it is unit-testable in isolation.
- **Information hiding:** clients call operations such as “add expense” and “get balances”; they do not know the storage schema.
- **Balances are never stored.** They are recomputed from expenses and settlements on demand, so they cannot drift out of sync with the underlying records.

6.2 Data Model

The core entities and their relationships are summarised below. Monetary columns hold integer paise. Foreign keys enforce that every share, expense and settlement belongs to a valid group and user.

Entity	Purpose	Key attributes and relationships
User	A registered person	id, name, email (unique), password hash, created-at
Group	A set of people sharing costs	id, name, creator (admin); has many memberships and expenses
Membership	Links a user to a group	user, group, role (admin or member)
Expense	A single payment made on behalf of the group	id, group, payer, amount (paise), description, category, date, split type, creator

Entity	Purpose	Key attributes and relationships
ExpenseShare	One participant’s portion of an expense	expense, user, share amount (paise); shares sum to the expense amount
Settlement	A payment between two members	group, from-user, to-user, amount (paise), timestamp
Activity	Audit-trail record	group, actor, action type, reference, timestamp

Table 9: Core entities

An expense has many shares; a group has many memberships, expenses, settlements and activity records; a user may belong to many groups. Because balances are derived, no balance column exists anywhere in the schema.

6.3 Core Algorithms

6.3.1 Money representation

All monetary amounts are stored as integers in paise. Rupee input such as 150.00 is converted to 15000 at the boundary and never handled as a floating-point number inside the system. Rounding occurs only inside the split-calculation services, and each split type has tests that prove the shares sum exactly to the stored amount.

6.3.2 Equal split

```
base      = floor(total / n)
remainder = total mod n
share(i)  = base + 1   for the first 'remainder' participants (ascending user ID)
          = base       for all others
```

Example: an expense of Rs. 100.00 (10000 paise) among 3 people gives 3334, 3333 and 3333 paise. The extra paisa goes to the participant with the lowest user ID. Because the order is fixed by user ID, the result is identical every time the calculation is run.

6.3.3 Exact split

Each participant’s share is entered by the user. The server validates that *sum(shares) = total* and rejects the request otherwise. No rounding is performed.

6.3.4 Percentage split

Each participant’s whole-number percentage is entered, and the server validates that *sum(percentages) = 100*. Each share is then computed as *floor(total × percentage / 100)*, and any leftover paise are distributed using the same remainder rule as the equal split.

```
Example: total = 10001 paise, percentages 33 / 33 / 34
  floor shares : 3300, 3300, 3400   (sum 10000)
  remainder    : 1 paisa -> lowest user ID
  final shares : 3301, 3300, 3400   (sum 10001)
```

6.3.5 Balance calculation

```
net_balance(member) = total_paid(member) - total_owed(member) +/- settlements
```

- A **positive** balance means the group owes this member.
- A **negative** balance means this member owes the group.
- **Invariant:** the sum of all net balances in a group is always 0. Every test that computes balances asserts this, so it acts as a test oracle.

6.3.6 Settlements

A settlement records that member A paid member B. It is treated as a payment that reduces A’s debt and B’s credit, so balances update without editing past expenses. The original expense history stays intact for

audit.

6.3.7 Debt simplification

```
1. Compute the net balance of each member.
2. Split members into debtors (balance < 0) and creditors (balance > 0).
3. Repeat until all balances are 0:
     a. Take the largest debtor and the largest creditor.
     b. Transfer min(|debtor|, creditor) from debtor to creditor.
     c. Update both balances and drop anyone now at 0.
```

Why greedy: it is simple, deterministic, and produces at most $n - 1$ transactions for n members with non-zero balances. Finding the true minimum number of transfers is NP-hard, so the greedy approach is a practical approximation that is easy to explain and verify.

6.3.8 Worked example

Four friends, A, B, C and D, take a trip. Three expenses are recorded, all with equal splits:

Expense	Paid by	Amount (Rs.)	Shared among	Each share (Rs.)
Hotel	A	2400	A, B, C, D	600
Food	B	900	A, B, C, D	225
Cab	C	600	A, B, C	200

Table 10: Expenses in the worked example

Member	Total paid	Total owed	Net balance
A	2400	600 + 225 + 200 = 1025	+1375
B	900	600 + 225 + 200 = 1025	-125
C	600	600 + 225 + 200 = 1025	-425
D	0	600 + 225 = 825	-825
Sum	**3900**	**3900**	**0 (invariant holds)**

Table 11: Balance computation

Debt simplification then produces three transfers: D pays A Rs. 825, C pays A Rs. 425, and B pays A Rs. 125. If D records a settlement of Rs. 825 to A, D's balance becomes 0 and A's drops to +550, and the remaining suggested transfers update automatically.

6.4 REST API Overview

The API follows REST conventions with JSON request and response bodies. Failures use a consistent envelope, `{ "error": { "code", "message" } }`. Validation problems return 4xx codes; unexpected failures return a generic 500 so internal details are never leaked to clients. The table lists the main resource groups.

Area	Representative operations	Access
Authentication	Register; log in and receive a JWT	Public (rate-limited)
Groups	Create group; list my groups; view group	Authenticated; members only
Members	Add a registered user by email	Group admin
Expenses	Create, read, update, delete expenses with split details	Members; edit/delete by creator or admin
Balances	Per-member balances and simplified transfers for a group	Members

Area	Representative operations	Access
Settlements	Record and list payments between members	Members
Activity	Chronological log of expense and settlement actions	Members
Dashboard	Total owed and owed-to-me across all groups	Authenticated user

Table 12: API resource groups (exact route paths are defined in the server source)

6.5 Security Design

- **Password storage:** bcrypt with per-password salt; plaintext passwords are never stored or logged.
- **Session management:** stateless JWT with a 7-day expiry, verified by middleware on every protected route.
- **Authorisation:** membership checks gate all group data; admin checks gate member management; creator-or-admin checks gate expense edit and delete.
- **Brute-force protection:** rate limit of 20 login or registration attempts per minute per IP.
- **Input validation:** Zod schemas reject malformed requests before they reach business logic.
- **HTTP hardening:** helmet sets secure response headers.
- **Error hygiene:** unexpected errors return a generic message; stack traces are not exposed.

6.6 Frontend Design

The React SPA is organised around the main user workflows: authentication, group list and group detail, expense entry, balances and settlement, activity feed, and the cross-group dashboard. Axios encapsulates API calls and attaches the JWT. Forms perform real-time validation (for example, showing a running total against the expected sum for exact and percentage splits) so that obvious mistakes are caught before submission, while the server remains the source of truth and re-validates everything.

---

7. Technology Stack

Layer	Choice	Justification
Frontend	React / Vite / Axios	Fast SPA and API integration; Vite gives quick builds and a rapid development loop
Backend	Node.js / Express	Small REST service with clear routes; one language across client and server
Validation	Zod	Declarative input contracts shared by routes and tests
Database	PostgreSQL / Prisma	Dockerized relational persistence; Prisma provides typed queries and migrations
Security	bcrypt / JWT / helmet	Password hashing and protected routes with hardened HTTP headers
Tests	Vitest / Supertest	Fast unit and API-oriented testing in the same toolchain
Deployment	Docker Compose	Reproducible environment for the app and its database

Table 13: Technology stack and justification

7.1 Why a Relational Database

Expense data is inherently relational — groups have members, expenses have shares, and shares reference users — and the balance invariant depends on consistent joins and transactional writes. PostgreSQL provides the integrity guarantees (foreign keys, transactions, constraints) that make the

invariant dependable, and Prisma provides a type-safe layer over it.

7.2 Deployment

The supported production setup is Docker Compose, which starts the application and a PostgreSQL 16 container together with consistent configuration. This removes “works on my machine” problems: every team member and the evaluator run the same environment. Local development without Docker is possible but requires manual environment-variable configuration and Node.js 20 or later.

```
# Quality commands (run from the repository root)
npm test        # run automated tests
npm run lint    # static code checks
npm run build   # production build
```

---

8. Implementation Progress

8.1 Backend Implementation

The backend of ExpenseEase is built using Node.js and Express, with a focus on modularity and scalability. Key backend features include:

1. Authentication and Authorization

- User registration and login are implemented with bcrypt for password hashing and JWT for session management.
- Role-based access control ensures that only group admins can add or remove members.

2. Expense Management

- CRUD operations for expenses allow users to create, read, update, and delete expenses.
- Split validation ensures that all expense shares (equal, exact, or percentage) are accurate and consistent.

3. Balance Calculation

- Balances are dynamically calculated from expenses and settlements, ensuring data consistency.
- Debt simplification uses a greedy algorithm to minimise the number of transactions required to settle debts.

4. Database Integration

- PostgreSQL is used as the relational database, with Prisma as the ORM for schema management and queries.
- Dockerized PostgreSQL ensures consistent development and production environments.

5. Testing

- The backend is thoroughly tested with Vitest and Supertest, covering unit tests, integration tests, and API tests.
- Test cases include edge scenarios such as invalid splits, unauthorized access, and concurrent updates.

8.2 Frontend Implementation

The frontend is a React-based Single Page Application (SPA) built with Vite for fast development and optimized builds. Key frontend features include:

1. User Interface

- A responsive and intuitive UI allows users to register, log in, create groups, add members, and manage expenses.
- The dashboard provides a summary of balances, activity logs, and group details.

2. Expense Workflow

- Users can add expenses with descriptions, amounts, categories, and split types.
- Real-time validation ensures that inputs are correct before submission.

3. Dashboard and Activity Log

- The dashboard displays “you owe” and “owed to you” balances across all groups.
- The activity log provides a detailed history of all expenses and settlements.

4. API Integration

- Axios is used for seamless communication with the backend API.
- Error handling and loading states are implemented to enhance the user experience.

8.3 Repository Structure

The repository is organised so that the application code and the academic deliverables are both easy to find.

```
ExpenseEase/
|-- client/                  React + Vite frontend
|-- server/                  Express API, Prisma schema, seed script, tests
|-- docker/postgres/         Database container configuration
|-- docs/                    SRS, design documents, diagrams, reports
|-- designs/                 UI design assets
|-- sample_data/             Seed accounts and expected results
|-- submission/              Organised mid-semester submission material
|-- Dockerfile, docker-compose.yml
|-- AI_USAGE_DISCLOSURE.md   Declaration of AI tool usage
|-- SUBMISSION_CHECKLIST.md  Submission checklist
|-- package.json             Root scripts (dev, test, lint, build)
`-- README.md
```

A seed script (`npm run seed`) creates local-only sample accounts, a group, expenses using all three split types, and a settlement, so the system can be demonstrated immediately and the expected results checked against the sample-data documentation.

8.4 Current Status

Area	Status
Backend	Fully functional with all core features implemented and tested
Frontend	Supports the complete registration-to-expense workflow, including group creation, member management, and expense recording
Testing	179 tests (157 backend, 22 frontend) have been successfully executed, ensuring high code quality
Quality gates	Lint and production build pass
Deployment	Docker Compose setup verified

Table 14: Implementation status

9. Testing Performed

9.1 Test Strategy

The testing strategy for ExpenseEase includes the following levels, chosen to match the layered architecture:

Level	Focus	Examples
Unit testing	Individual functions and modules in isolation	Split validation, rounding remainder, balance calculation, debt simplification
Integration testing	Interaction between modules, such as API endpoints and the database	Create expense then read balances; record settlement then re-check balances
End-to-end testing	Real user workflows through the whole system	Register, create a group, add members, record expenses, settle

Table 15: Testing levels

Verification versus validation. Requirement reviews, code reviews and automated tests check that the system was built correctly (verification). Demonstrations and walkthroughs with the seeded data check that the right system was built (validation).

9.2 Test Categories and Oracles

- **Split correctness:** for every split type, shares must sum exactly to the expense total; invalid sums are rejected with a clear error.
- **Determinism:** repeated calculation on the same input yields identical output, including which participants receive leftover paise.
- **Balance invariant:** the sum of all net balances in a group is zero; this is asserted across scenarios and acts as a test oracle.
- **Simplification:** after applying all suggested transfers every balance is zero, and the number of transfers never exceeds $n - 1$.
- **Authentication:** valid login succeeds; invalid credentials, missing token and expired token are rejected.
- **Authorisation:** non-members cannot read a group; non-admins cannot add members; only creator or admin can edit or delete an expense.
- **Edge cases:** invalid splits, unauthorised access, and concurrent updates.
- **Frontend:** component rendering, form validation and API integration behaviour.

9.3 Test Results

On 30 September 2026, the following tests were executed:

Suite	Number of tests	Scope	Result
Backend	157	Authentication, group management, expense CRUD, split validation, balance calculation, debt simplification	All passed
Frontend	22	UI components, form validation, API integration	All passed
Total	**179**		**All passed**

Table 16: Test results, 30 September 2026

9.4 Evidence

The `npm test` command output confirms that all tests passed without errors. Linting (`npm run lint`) and build (`npm run build`) were also successful. These three commands form the quality gate that must pass before changes are merged to the main branch.

---

10. Project Management Status

10.1 Development Approach

The project follows an iterative and incremental SDLC. Each increment delivers a working subset of features, which is then tested and reviewed before moving to the next increment. This approach ensures that the system is always in a deployable state.

Prototyping was used first for the workflows whose requirements were unclear (group creation, split entry and the settlement flow) so that feedback could be collected before the requirements were fixed. Incremental delivery was then used for the build. The hybrid suits this project because the splitting rules were well understood while the user-interface flow was not, early working software was needed, and the system divides naturally into modules.

10.2 Completed Increments

Increment	Scope	Key deliverables
1. Authentication and Group Workflows	User registration, login, JWT authentication; group creation and member management	Auth API, group API, registration/login UI, group UI
2. Expense and Split Workflows	Expense CRUD with equal, exact and percentage splits; split validation	Expense API, split services, expense UI
3. Balance, Settlement and Dashboard Workflows	Dynamic balance calculation, debt simplification, settlement recording, activity logs, cross-group dashboard	Balance/settlement APIs, dashboard and balance UI

Table 17: Completed increments

10.3 Work Breakdown Structure

```
ExpenseEase
|-- Requirements and analysis (SRS, user stories, use cases)
|-- Design (architecture, data model, interfaces)
|-- Implementation
|   |-- Auth and groups
|   |-- Expenses and splits
|   |-- Balances and settlements
|   `-- Activity feed and dashboard
|-- Quality (tests, lint, reviews)
`-- Delivery (docs, README, Docker verification, demo)
```

10.4 Estimation and Scheduling

Effort was estimated with two complementary methods so the figures could be cross-checked:

- **Function Points:** count external inputs, outputs, inquiries, internal files and interfaces, apply complexity weights to obtain unadjusted function points (UFP), then multiply by the technical complexity factor, $TCF = 0.65 + 0.01 \times DI$.
- **Basic COCOMO (organic mode):** $Effort = 2.4 \times (KLOC)^{1.05}$ person-months and $Tdev = 2.5 \times (Effort)^{0.38}$ months.

Activities were sequenced with dependencies, and durations were estimated with three-point (PERT) estimates, *te = (O + 4M + P) / 6*. The critical path was found with forward and backward passes; the network diagram is kept in the docs folder.

10.5 Team Contributions

Member	Tasks completed	Share
Shreeya Srivastava	Group and membership integration, group UI, tests, documentation review	25%
Shraddha Sharma	Authentication API, JWT integration, registration/login UI, tests, README review	25%
Rudransh Pratap Singh	Expense APIs, split validation, expense/activity UI, tests, test-case review	25%
Sachit Jain	Balance/debt/settlement APIs, dashboard/balance UI, Docker verification, demo preparation	25%

Table 18: Team contributions

10.6 Risks and Mitigation

Risk	Impact	Mitigation Strategy
Incorrect balance calculations	Wrong amounts owed; loss of trust	Automated tests ensure accuracy; integer paise; deterministic remainder rule; balances dynamically computed; invariant tests
Unauthorized access	Exposure of group financial data	Role-based access control and JWT authentication are enforced on every endpoint
Deployment issues	System fails to start in a new environment	Dockerized setup ensures consistency across environments
Changing requirements	Rework and schedule slip	Incremental delivery and a controlled change process
Team availability	Delayed modules	Small increments, shared ownership of modules, cross-review of tests
Integration failures	Late-discovered defects	Incremental integration; tests run on every change

Table 19: Risk register

10.7 Configuration and Change Management

- Git is used for version control with small, focused commits; the repository history exceeds one hundred commits.
- Requirement changes are recorded with the reason, impact and approval, and the SRS and traceability links are updated accordingly.
- The main branch is kept in a buildable and testable state at all times.
- AI-assisted work is declared in AI_USAGE_DISCLOSURE.md, and all such content was verified and modified by the team.

11. Challenges Faced and Solutions Attempted

Challenge	Description	Solution
1. Deterministic rounding	Ensuring that leftover paise are distributed fairly and repeatably in equal and percentage splits	Allocated leftover paise in ascending user ID order, so the same input always produces the same shares
2. Debt simplification	Reducing the number of transactions required to settle debts without an exponential search	Implemented a greedy algorithm that matches the largest creditor and debtor, bounding transfers at n − 1
3. Authorization	Preventing unauthorized access to group data and operations	Enforced membership and admin checks for all sensitive operations
4. Balance consistency	Avoiding stale or incorrect balance data	Calculated balances dynamically from expenses and settlements; never stored them

Table 20: Challenges and solutions

11.1 Reflections

The most instructive challenge was rounding. Handling money in integer paise removes floating-point error, but it does not remove the question of who absorbs a leftover paisa. Making that choice explicit, deterministic and tested turned a vague source of disputes into a verifiable rule. Similarly, choosing to derive balances rather than store them removed an entire class of synchronisation bugs and made the zero-sum invariant straightforward to assert.

A second lesson was the value of tests as a design tool. Writing the invariant and rounding tests first clarified the interfaces of the service layer and encouraged the separation of business logic from HTTP handling that now makes the code easy to maintain.

12. Remaining Work

- Conduct a final round of usability testing with real users to identify and fix any UX issues.
- Optimize the performance of the frontend and backend for faster load times and API responses.
- Add a feature to export group expense summaries as a PDF for offline sharing.

12.1 Possible Future Extensions

Beyond the immediate remaining work, the exclusions listed in Section 3.2 form a natural roadmap: email-based password reset, notifications for new expenses and settlements, receipt attachments, multi-currency support, recurring expenses, group archiving, and an installable progressive web app with offline support. Each can be added without altering the core balance model, because balances remain derived from the underlying expense and settlement records.

12.2 Conclusion

ExpenseEase meets its central objective: giving a small group a single, honest ledger whose numbers anyone can verify. The backend is fully functional, the frontend supports the complete workflow, and 179 automated tests together with lint and build checks pass. Equally important, the project has been run as an engineering exercise — with explicit requirements, a layered design, traceability from need to test, a managed risk register and an incremental delivery plan — which is the practice this course set out to teach.

13. References

1. **Prisma Documentation** — used for database schema management and queries.
2. **Express Documentation** — used for building the backend REST API.
3. **React Documentation** — used for developing the frontend Single Page Application (SPA).
4. **Ian Sommerville, Software Engineering** — referenced for applying software engineering principles like SDLC and requirements validation.
5. **Course lecture material** — used for understanding life cycle models, requirements traceability, and software testing.

Appendix A. AI-Use Disclosure

See AI_USAGE_DISCLOSURE.md in the repository. Students must verify and modify AI-generated content as needed and remain responsible for correctness, security, and originality.

Appendix B. Glossary

Term	Meaning
Paise	One hundredth of a rupee; the integer unit in which all amounts are stored
Net balance	Total paid minus total owed, adjusted for settlements; positive means owed to the member
Balance invariant	The sum of all net balances in a group is always zero
Settlement	A recorded payment between two group members
Debt simplification	Reducing the set of debts to the fewest practical transfers
JWT	JSON Web Token; a signed token that proves a user’s identity on each request
bcrypt	A salted, adaptive password-hashing function
SPA	Single Page Application; a web app that updates in place without full page reloads
RTM	Requirements Traceability Matrix; links needs to requirements, design and tests
SDLC	Software Development Life Cycle